Client-Server Chat Application

A real-time desktop chat application built using **Python**, **TCP Socket Programming**, **Multithreading**, and **Tkinter**. The project demonstrates client-server communication over a network while providing a simple graphical interface for sending and receiving messages.

Features

- Real-time messaging between client and server
- TCP socket-based communication
- Separate client and server applications
- Tkinter-based graphical user interface
- Concurrent message handling using multithreading
- Reliable connection establishment and message transmission
- Modular and easy-to-understand project structure

Technologies Used

- Python 3
- Socket Programming (TCP)
- Multithreading
- Tkinter (GUI)

Project Structure

```
Client-Server-Chat/
├── client.py          # Client application
├── server.py          # Server application
├── README.md
```

How It Works

1. Start the server application.
2. Launch the client application.
3. The client establishes a TCP connection with the server.
4. Messages are exchanged in real time through the graphical interface.
5. Separate threads handle incoming messages, ensuring the GUI remains responsive.

Installation

1. Clone the repository:

```
git clone https://github.com/your-username/client-server-chat.git
```

1. Navigate to the project directory:

```
cd client-server-chat
```

1. Ensure Python 3 is installed.

Running the Application

Start the Server

```
python server.py
```

Start the Client

```
python client.py
```

> **Note:** If running on different devices, update the server IP address in the client application to match the server's local IP.

Future Improvements

- Multiple client support
- User authentication
- Private messaging
- Chat history storage
- File sharing
- End-to-end encryption
- Emoji support
- Cross-platform executable build

Learning Outcomes

Through this project, I gained practical experience in:

- TCP socket programming
- Client-server architecture
- Multithreaded programming
- Desktop GUI development with Tkinter
- Network communication and debugging
- Writing modular Python applications

Screenshots

Add screenshots of:

- Server window
- Client window
- Chat conversation
- GUI interface

License

This project is intended for educational and learning purposes.